P1683R0
References for Standard Library Vocabulary Types - an optional<> case study

Published Proposal, 2020-02-07

Author:
JeanHeyd Meneide

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:
https://thephd.github.io/vendor/future_cxx/papers/d1683.html

Reply To:
JeanHeyd Meneide, @ThePhD

Abstract

For over a decade now, the question of how to handle references in Standard Library types has left several standard vocabulary types in a severe limbo. Whether it is std::optional, std::variant, or upcoming abstractions such as std::expected, there is a constant question of how one should handle the fundamental reference types that pervade C++ code. This paper surveys the industry of references in complex wrapper/composite types using user surveys, code review, field experience, and more to inform the conclusions. The paper explores this design space with the canonical composite wrapping type, optional.

Table of Contents

1. Revision History

1.1. Revision 0 - February 7th, 2020

— The only release, and the only revision. Ever.

2. Motivation, Goals and History

Originally, `std::optional<T>` -- where `T` denotes a type -- contained a specialization to work with references, `std::optional<T&>`. When some of the semantics for references were called into question with respect to the assignment operator (assign into the value or rebind the reference inside the `optional`),the comparison operators (based on `operator<` exclusively or forwarding all operations directly to the underlying type), and the choice of "deep const", the debate became philosophically deadlocked. For years, no full consensus was reached. In the end, removing the entirety of `std::optional<T&>` gave way to some consensus, and comparisons for the value `T` forwarded to the underlying type, if present. This left many codebases in an interesting limbo. Previous implementations and external implementations had support for references, while the standard did not. Advocates vigorously spread the word of two equally valid and hard to pick interpretations for the assignment operator, making it the canonical example of failure to implement.

This problem became apparent with other parameter and return abstractions such as `std::expected`, the Boost Outcome library, `std::variant`, and more. Given the scope of the problem, qualitative research was necessary to help capture the uses and intent of references in vocabulary types. `optional` was chosen as the primary candidate to collect data, since it is widely used and reimplemented in the C++ ecosystem and broadly represents the challenge of reference wrapping types in C++. A survey conducted of 110+ programmers, (private) feedback from over a dozen companies, and several analyses of optional reference usage (or not) in the wild has yielded results as to quality of implementation, existence, and prevalence of references and their use.

Pointedly: there is strong motivation to have references in base vocabulary types used for both parameters and returns (variant, optional, expected, future, etc.). A lot of existing practice to do so both before and after the standard settled on its current semantics for `std::optional` proved out particular designs in many different industries. Brief historical analysis and user communication reveals the Standards Committee actually had a "chilling effect" on reference support in implementations in large codebases, from private companies to open source projects.

Finally, given the field experience and absolutely lack of implementation experience with certain models as well as contradictions with the fundamental elements of C++, this paper makes a recommendation for one of the models in hopes of moving discourse out of its current deadlock and toward a friendly, sustainable solution for the C++ ecosystem.

2.1. Stuck in the Past

A very large hole is left in many codebases that desire to wrap their non-null optional returns from `T*` to `T&`. It has prevented many code bases from migrating from the most popular optional implementations available pre-standardization, such as akrzemi/optional and Boost.Optional. It has also prevented adoption in other modern codebases for where the

difference between `T*`, `optional<T&>`, and `optional<std::reference_wrapper<T>>` alongside programmer intent is significant and non-ignorable, especially in the case of porting code that used to use `boost::optional`. Many of these programmers have decided to either take the painful route of transitioning, or to simply declare it a non-starter and just use `boost`.

This has forced many library authors in need of a vocabulary "optional" to have to add preprocessor-based switches to use different kinds of optional implementations and has drastically increased implementation burden. For a vocabulary type, `optional` contains an incredibly high fragmentation of implementations: some implementations are modeled for easy porting to the standard, but many still have custom support not found in the standard (void specializations, reference handling, monadic functions, and more). Implementation quality varies quite a bit among available optionals, supporting various standards, exception modes, trivial/explicit propagations, reference support, void support, `constexpr` capabilities, and more.

Eliminating the need for this by including common goals -- standard goals -- would greatly benefit both library developers and the ecosystem at large with which they interact. We cannot get to these common, standard goals if the landscape is not deliberately and methodically explored, hence this paper.

§ 2.2. Surveying the Present

It has been nearly a decade since `std::optional` was slated to end up in the standard, even if it only reached the International Standard in C++17. Now that C++ has come this far, this paper is going to take a survey of the landscape and of the many implementations of `optional` in order to analyze use cases and experience.

In furthering this goal, a survey of developers from all experience levels and professional/hobbyist tracks was taken. While there are public implementations of `std::optional` in various flavors and names, it is also important to capture private interests with similar types. Several e-mails were sent out as well, and this proposal will attempt to succinctly describe both those and the survey. The e-mails are kept anonymous and confidential (as that is the condition upon which this paper accepted private communications in order to assuage the concerns of employees/employers). This is mostly to protect the innocent and be careful.

Furthermore, the C++ ecosystem has not had an honest evaluation of the design space since the conception of the original optional papers and Nevin's incredible work for `std::variant`. This paper seeks to rectify this.

§ 2.3. Impact for the Future

The question of references being used in vocabulary types is not just for `optional`: all sorts of basic types where references might find their way in because they serve as a family of wrappers/composite/"transportation" types such as `std::variant`, `std::tuple`, `std::expected` and others. The decision here will rest of other vocabulary types, *except* the case of `std::tuple` where C++'s fate has already been decided by `std::tie`.

The recommendation presented further along in this paper should be extended to the other necessary vocabulary types. Not doing so risks the same degree of questionable design choices for these other types and further indecision that leads to a permeation of implementations that try to do the same things but are subtly incompatible or not very well implemented. Already, `std::expected` is seeing a small degree of implementation churn on the outside (not quite as much as `optional`) in Boost.Outcome, LEAF, `tl::expected` and more; the community is trotting out the same story over and over again for basic, fundamental types of similar flavor. This should be done once, properly, and solved until new information is presented.

§ 2.4. Fragmentation

As mentioned previously, another key motivation of this paper is the surprising amount of fragmentation that exists in the C++ community regarding the `optional` to use. It is an incredibly poor user experience to have several types which perform fundamentally the same operations but do not cover the needs of the vocabulary type that have been demonstrated by codebases for well over a decade now. At least 35 public and private implementations (many of which are listed and referenced in this paper) with varying levels of conformance, performance, and design goals. What is even more troubling is that users continue to roll their own optionals to this day, even on C++17 compliant compilers or standard libraries (e.g.,

with `std::optional` being available). Dissatisfaction with the optional provided by the standard library and its lack of features deemed useful by the broader family of C++ programmers means that in some manner the current optional has failed to meet the needs and expectations of the programmers who are both coming to C++ and the programmers who have worked in C++ with boost or their own company codebase for a long time.

§ 3. Design Considerations

This paper reviews implementation experience, models, and theory around what a composite / wrapper types like `optional`, `variant`, and `expected` should do. This paper also dives into a survey of 110+ developers (plus a few additional members who espoused their opinions directly VIA e-mail, instant messaging mediums, Twitter, and elsewhere) to understand what is necessary in the optionals they use in real-world projects, company projects, hobby projects and more. The survey results are left in raw form at this location for anyone who wants to peruse them.

§ 3.1. The Great Big Table of Behaviors

Below is a succinct synopsis of the options presented in this paper and their comparison with known solutions and alternative implementations. It does not include the totality of the optional API surface, but has the most exemplary pieces. A key for the symbols:

✔️ - Succeeds

🚫 - Compile-Time Error

❌ - Runtime Error

❓ - Implementation Inconsistency (between engaged/unengaged states, runtime behaviors, etc.)

optional behaviors					
Operation	**T**	**std::reference_wrapper<T>**	**T& conservative**	***Recommended:* T& rebind**	**T**
exemplary implementation(s)	✔️ std::optional nonstd::optional llvm::Optional folly::Optional	✔️ std::optional nonstd::optional llvm::Optional folly::Optional	✔️ std::experimental::optional sol::optional	✔️ boost::optional tl::optional ts::optional_ref	🚫 ...?
`optional(const optional&)`	✔️ copy constructs `T` (disengaged: nothing)	✔️ binds reference (disengaged: nothing)	✔️ binds reference (disengaged: nothing)	✔️ binds reference (disengaged: nothing)	✔️ bind (dise noth
`optional(optional&&)`	✔️ move constructs `T` (disengaged: nothing)	✔️ binds reference (disengaged: nothing)	✔️ binds reference (disengaged: nothing)	✔️ binds reference (disengaged: nothing)	✔️ bind (dise noth
`optional(T&)`	✔️ (copy) constructs `T`	✔️ binds reference	✔️ binds reference	✔️ binds reference	✔️ bind
`optional(T&&)`	✔️ (move) constructs `T`	🚫 compile-time error	🚫 compile-time error	🚫 compile-time error	🚫 com erro
`operator=(T&)` *engaged*	✔️ overwrites `T`	✔️ rebinds data	🚫 compile-time error	✔️ rebinds data	✔️ over
`operator=(T&)` *disengaged*	✔️ overwrites data	✔️ rebinds data (overwrites reference wrapper)	🚫 compile-time error	✔️ rebinds data	✔️ rebi
`operator=(T&&)` *engaged*	✔️ move-assigns `T`	🚫 compile-time error	🚫 compile-time error	🚫 compile-time error	🚫 com

					erro over
`operator=(T&&)` *disengaged*	✔ constructs `T`	🚫 compile-time error	🚫 compile-time error	🚫 compile-time error	🚫 c com erro sher
`operator=(T&)` *engaged*	✔ overwrites `T`	🚫 compile-time error	🚫 compile-time error	✔ rebinds pointing reference	✔ over
`operator=` `(optional<T>&)` *disengaged*	✔ overwrites data	✔ overwrites data	🚫 compile-time error	✔ rebinds pointing reference	✔ rebi
`operator=` `(optional<T>&&)` *engaged;* *arg engaged*	✔ move assign `T`	✔ rebind data	✔ rebind data	✔ rebind data	✔ mov
`const` propagation on `.value()`	✔ propagates - deep	✔ shallow	✔ shallow	✔ shallow	✔ prop
`operator=` `(optional<T>&&)` *engaged;* *arg disengaged*	✔ disengage `T`	✔ disengage `T`	✔ disengage `T`	✔ disengage `T`	✔ dise
`operator=` `(optional<T>&&)` *disengaged;* *arg disengaged*	✔ nothing	✔ nothing	✔ nothing	✔ nothing	✔ noth
`*my_op = value` *engaged*	✔ copy assigns `T`	✔ copy assigns `T`	✔ copy assigns `T`	✔ copy assigns `T`	✔ copy
`*my_op = value` *disengaged*	❌ runtime error	❌ runtime error	❌ runtime error	❌ runtime error	❌ runt
`*my_op =` `std::move(value)` *engaged*	✔ move assigns `T`	✔ move assigns `T`	✔ move assigns `T`	✔ move assigns `T`	✔ mov
`*my_op =` `std::move(value)` *disengaged*	❌ runtime error	❌ runtime error	❌ runtime error	❌ runtime error	❌ runt
`(*my_op).some_member()` *engaged*	✔ calls `some_member()`	🚫 compile-time error	✔ calls `some_member()`	✔ calls `some_member()`	✔ calls som
`(*my_op).some_member()` *disengaged*	❌ runtime error	❌ runtime error	❌ runtime error	❌ runtime error	❌ runt

§ 3.2. Problems in the Design Space

As hinted at in the the table, there are numerous design tradeoffs and some that just have outright problems with various optional implementations.

§ 3.2.1. Problems: Assignment Semantics

Assignment semantics produces a level of fervor and almost religious backlash to even discussing `std::optional` and references. The original contention rose out of a purported contention between the semantics. That is, given the following snippet:

```cpp
#include <optional>
#include <iostream>

int main (int, char*[]) {
    int x = 5;
    int y = 24;
    std::optional<int&> opt = x;

    opt = y;
    std::cout << "x is " << x << std::endl;
    std::cout << "y is " << y << std::endl;

    return 0;
}
```

The contention is the "two plausible interpretations". Under what is known as "Rebinding", the following results show:

```
x is 5
y is 24
```

for what is known as "Assign-through", the following results show:

```
x is 24
y is 24
```

This has become an explosive topic in the Committee and the C++ Community in general and is a heavy crux of contention that, unlike comparison operator specification, could not be resolved before the final shipping date of the current `std::optional`.

§ 3.2.2. Problem: Status Quo, but Monadic?

The highest feature request for optional from the conducted survey is monadic operations. This proposal does not go into it: it is detailed in Sy Brand's [p0798]. However, it is critical to note that many of these chained monadic operations cannot be implemented efficiently without some form of reference handling through the optionals. Lots of implicit and uncontrolled copies can result from long chains of `and_then` and `map` that propagate optionals through, resulting in poor performance unless the user explicitly inserts glue code that handles raw references and returns them as `std::reference_wrapper`s and similar. This deeply impacts code wherein the Builder Pattern is used, or functions that self-modify the object in question before returning a reference to `self`. Given that this is a common idiom, this design decision leaves much to be desired in the pool of discourse.

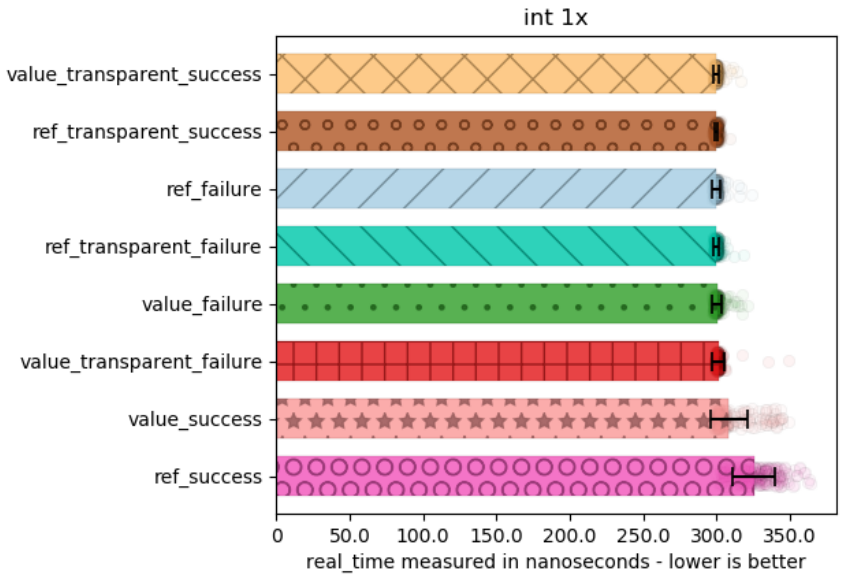
Quantifiable performance impact is also easy to achieve. While code using small value types like `int` can mostly avoid this, structures which have more complicated copy and move semantics like `std::string` and `std::vector` suffer noticeable and tractable performance problems, even at miniscule sizes like a `vector` of 8 `int`s.

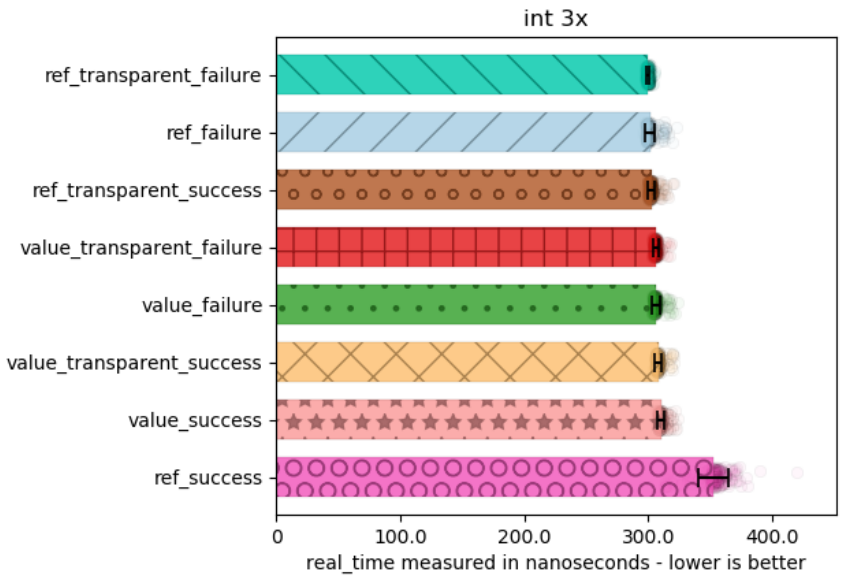
The full code is buildable with CMake and runnable with a pretty graph-visualizer. There are a few things tested here. First, the basic case is tested: cheap values that can fit in registers (integers, basically). Then, tests for another common type are done: `std::vector<int>`. Nothing special is put in the `vector` except integers, because that enables the `memcpy` optimizations (and lower general overhead to what is being measured) where possible. As another data point, comparisons between 1 monadic operation piping versus 3 monadic operation piping is checked for each type. Finally, for all of these cases, code is tested when it is completely visible to the compiler (marked with **_transparent** in the name). This means it is available for inlining. Conversely, the code is also tested for when it is completely invisible / opaque to the compiler. Anything without **_transparent** had its transforming work call hidden behind a DLL call. This provides clear controls for the benchmark with respect to inlining or RVO.

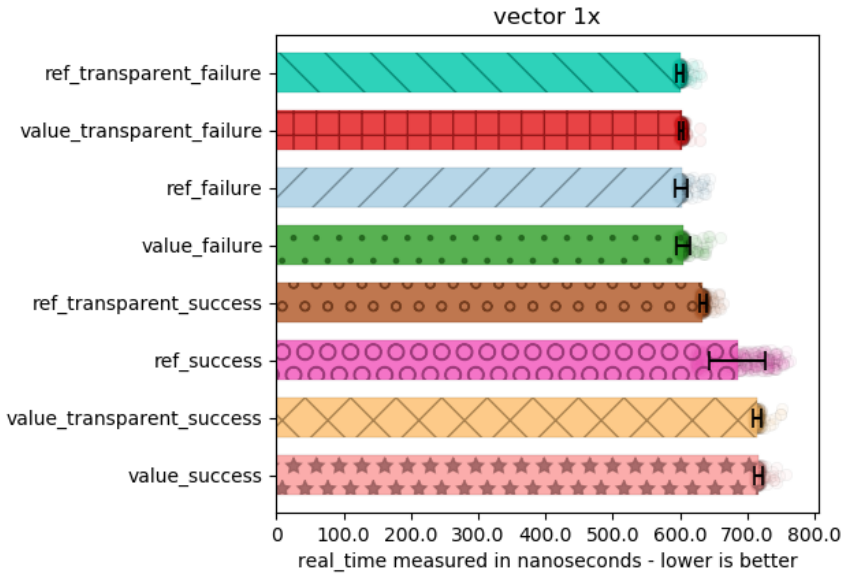
As a slight addendum and a fail-safe to test if the code went wrong, there are also two sections: one is marked **_failure**, and this just means that the optional was empty so no work should have been done in the monadic functions. It's essentially a

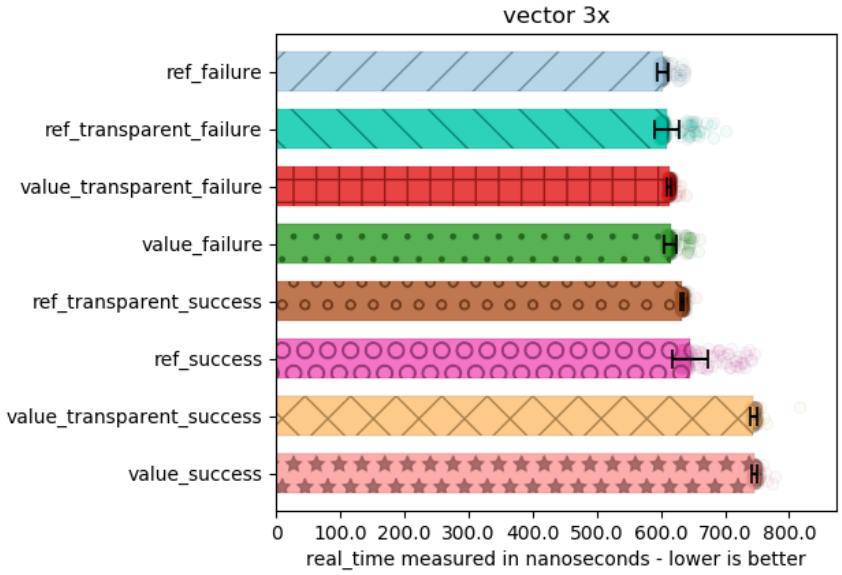
basic litmus test for “is something haywire happening in the benchmarks right now?”. There is also **_success**, where the optional was filled with a value and everything worked out.

As part of the benchmark, computed values are checked for legitimacy, to make sure the optimizer doesn’t just toss everything out on us. Our transformation is essentially just multiplying every element in the `vector` by `2`, then returning the result.









As demonstrated can see, even for tiny vectors, mapping once versus mapping three times produces a noticeable performance impact. This means today's optimizers -- in a case where the compiler can see the entire tiny function ("transparent") or not -- cannot elide the additional moves and copies in a value-based world. This has significant performance implications for applications wishing to take advantage of Sy Brand's excellent work for monadic operations and throws some potential shade on other monadic-capable types such as `variant`, `expected`, and more. This may also produce some noticeable problems for people attempting to work with pattern matching mixed with this feature in the future, albeit implications for such are presently unclear and speculative at best.

§ 4. Implementation/Deployment Experience

§ 5.1 Recommendation: Rebinding, shallow const, deep comparison reference types has long-standing design, implementation, and industry experience with several high-quality implementations and a handful of conference speakers's talks, as well as significant research put into them. § 4.2 Assign-Through Deployment and Experience has not been reported

to see much experience (any experience, currently), other than the author's anecdotal experimentation as a beginning programmer, further experimentation before this proposal, and a single other Hobby Developer's long-term C++ project.

§ 4.1. Conservative and Rebinding

The "conservative solution" proposed in [p1175r0] is a much more tame version that has seen implementation experience for at least 6 years in akrzemi/optional, and 4 years in sol2 (albeit the implementation for sol2 has been changed to instead mimic the full "complete" version described by § 5.1 Recommendation: Rebinding, shallow const, deep comparison reference types, migrated over with no complaints or problems with users). The simple version has also existed as the advised compiler-safe subset for Boost.Optional for 15 years, maybe more (this was mostly from warning about the inability of pre-C++11 to delete or restrict r-value bindings and how `const T&` could bind to more than was likely intended outside of parameters).

The "complete" version has seen implementation experience for even longer for those who used the full functionality of Boost.Optional. It is also present in Sy Brand's optional, a number of industry optionals, and the author's independent optional. The boost mailing list thread on this topic indicates much of the same potential confusion around references, but towards the end there was the realization that due to inconsistencies with how assign-through behaves the behavior of assign-through is far from ideal.

Jonathan Müller's type_safe has it, but under a different name (`optional_ref`). From Müller's C++Now 2018 'Rethinking Pointers' Talk, it is easy to see why: he argues that using a type which very explicitly demarcates its purpose with a name (`optional_ref<T>` as compared to `optional<T&>`) is better for an API interface. This works just as fine as any other argument, until the case of generic programming comes around. This is where the difference between `optional_ref` and `optional` as 2 distinctly named, strong types becomes noticeable and painful for any given developer. The library developer can add a level of indirection as mentioned in Tristan Brindle's musings for Optional References, but this is similar to the awkwardness of having to explicitly `unref_unwrap<>` every type just in case `std::reference_wrapper` shows up in generic code. Every time you might expect to be dealing with `optional` and `optional` references, a developer has to furnish a number of metaprogramming tools and once more add special handling to functions and classes that should never had to think about these things.

The explicit `optional_ref` methodology can be used as a way forward but costs generic programmers a useful vocabulary type and makes type duplication for reference categories a severe issue. For example, while this duplication works for `optional<T>` (`optional_ref<T>`), what happens to `variant<T...>` (`variant_ref<T...>` ??) or `expected<T, E>` (`expected_ref<T, E>` ??)? A variant would easily want to mix reference and value types for many, many reasons: the `optional_ref` solution, while targeted and useful, does not scale as a generic and workable solution for either experts or beginners in need of more.

§ 4.2. Assign-Through Deployment and Experience

This variation of the recommended design found below is an assign-through `optional` that attempts to mimic the semantics of a reference as much as the `optional` specification allows it. This includes assign-through behavior when the `optional` is in an engaged state.

Assign-through as a design is harmful to programmers due to its change-of-behavior depending purely on a runtime property (engaged vs. unengaged). As shown in § 3.1 The Great Big Table of Behaviors, many of the other options are much more consistent and have much less questions surrounding the totality of its behavior.

Most notably is the lack of decided semantics for what happens in the engaged state versus the unengaged state with assignment. This version of an `optional<T&>` would have to rebind for assignment operators that take a right hand side of a l-value. Still, the more dangerous case is the r-value case. `int a = 24; int& a_ref = a; a_ref = 12;` is valid code. Many proponents of `std::optional<T&>` who read it solely as an extension of references as they are in the language demand syntactic equivalence with references for "consistency" reasons. If syntactic purity is the goal with respect to references here, then assignment of optionals would have to tolerate this use case.

If r-value assignment is not a compiler error, then the unengaged state with r-value assignment is left, of which there is no good answer: do we throw `bad_optional_access`? `terminate` so it can be `noexcept`? These answers and many more do not seem to adequately address the problem space for which these types may be used (potentially deferred returns, return types from lookup operations, and optional parameters), and other decisions like it are increasingly questionable in either

utility or usefulness. Having optional references behave as the reference they contain is a poor design choice: trying to gloss over the fact that they are, indeed, optionals (nullability for any type) results in a poor quality abstraction that is hard to use, harder to reason about at compile and run time, and bug-prone.

Specifically, with respect to its bug-prone nature, it is effortless to write code using an assign-through optional where something that may or may not be a code smell cannot be detected simply by looking at the code. Consider the following snippet, adapted from [foonathan-optional-problems]:

```
int foo_bad(int x, optional<int&> maybe_y) {
        int value = 40;
        /* lots of code */
        maybe_y = value;
        /* lots of code */
        return 24;
}
```

In an assign-through world, this code is *valid* because `maybe_y` might *actually* have something inside of it. In the engaged case, it will assign to whatever was previous bound inside of `maybe_y`. This means that you can be asking for dangling reference problems based purely on the engaged vs. unengaged states.

Static analysis tools cannot definitively point to this code as bad: the best it can offer is a warning, because there exists runtime states where this is exactly what the programmer intended to do. It also creates insanely hard to track bugs that are only discoverable under valgrind or ASan-like tools. Undefined behavior because a manifestation of a collection of runtime properties rather than a mistake that can be caught immediately by tools or by the eyes of a code reviewer is the front-running poster child for hard to track "Heisenbugs".

Furthermore, things that neither humans nor static analysis can properly diagnose come from code that looks and behaves innocently. Consider a sparse `cache` class which uses a default resource most of the time, but upon finding a proper value takes different behavior:

```
std::optional<int&> cache::retrieve (std::uint64_t key) {
        std::optional<int&> opt = this->get_default_resource_maybe();
        // blah blah work
        auto found_a_thing = this->lookup_resource(key);
        if (found_a_thing) {
                int& resource =
                  this->get_specific_resource(found_marker);
                // do stuff with resource, return
                opt = resource;
        }
        return opt; // optional says if we got something!
}
```

This code works perfectly fine most of the time, except in the case where it actually finds something related to `key` and a default resource is present. In this case, it will **overwrite the default resource**. This is a severe problem because the optional object is still the same: nothing about it has changed, but everything about the original object has changed before getting returned. No amount of static analysis will tell you this is a mistake, as it is a logic bug induced by a mental model of the optional that constantly rebinds when `opt` is empty, but assigns-through in the one case where it needs to assign twice over.

These are all extraordinarily dangerous developer traps waiting to happen with assign-through optionals.

While there is some theory crafting around assign-through optionals and variants, it has zero publicly available implementation or deployment experience, nor any private industrial support as far as the survey shows. Only 2 out of 110 survey responses report using an assign-through optional. Neither point to a repository. One is used for projects and hacks, the other is used for a large company project. It is notable that for the individual who reported using an assign-through optional in a company project, there was no firm conviction behind the code: it is "a lazy implementation" that predates boost, and has not actually had a reference put inside of it ever. (In other words, it does not count as implementation experience.) The only other user to have an assign-through optional in their projects put rebinding optionals on their wish list: the only people that appear to want an assign-through optional are developers that never, ever implemented or used one.

Asides from these two survey respondents, many companies, Boost Users, the Twitter-verse, several Discord servers, the CppLang Slack, and many more e-mails to C++ programmers across the globe probed for real significant use of an assign-through optional. Nobody has reported using a non-rebinding or non-conservative optional solution in their code base to date, or if they do they are not aware of it and have not given this information in the last year for some reason.

This leaves a serious question of the validity and usefulness for assign-through. It may be that in publishing Revision 0 of this paper, individuals who the author could not reach directly or by survey will come out to inform the author of a non-rebinding reference optional that has seen experience and use as a Studio, Company, and/or shop across the globe. The author encourages everyone to please submit actual deployment experience.

However, given the above, assign-through appears to be exactly that: a fanciful unicorn that does not exist except for the sole purpose of creating unnecessary and directionless bikeshed. It is a trap that masks itself in the clothes of syntactic similarity with references while having demonstrably harmful properties that cannot stand up to even basic design principles for Modern C++'s generally thoughtful and bug-proofing abstractions. It represents a foolish consistency for consistency's sake and there should not be a future in which it exists for C++, whether that is C++20 or C++50.

Similarly, a "deep const" model does not fit for the same reasons stated above.

§ 4.3. Rebind, but Pointer Comparisons

Rebinding with the addition that comparison only compares pointers -- ignoring unspecified behavior of pointer comparisons when not using `std::less` -- has some design experience. This is based on the idea that the salient properties of an optional -- in this case, what gets worked on in the constructor and assignment -- must also be what is used for comparison. These qualities are explored by Matt Calabrese in his library Argot.

The author appreciates this model but notes that this is extremely similar to Java's model for how types should behave. Pointers are used for comparisons in references always, and to introduce a pointer-oriented comparison ideology here would not only inconsistent but useless to the point of pushing C++ developers to have to define a special `is_equal` method every single time they wish to do comparison between references. To illustrate the problem a little more effectively:

```
template <typename T>
bool maybe_comp (optional<T> left, optional<T> right) {
        return left == right;
}
```

Under the Argot model:

```
int x = 5;
int y = 5;
int& x_ref = x;
int& y_ref = y;
optional<int> maybe_x = x;
optional<int> maybe_y = y;
optional<const int&> maybe_ref_x = x_ref;
optional<const int&> maybe_ref_y = y_ref;

bool r0 = x == y;
bool r1 = maybe_comp(maybe_x, maybe_y);
assert(r0 == r1); // assertion passes

bool ref_r0 = x_ref == y_ref;
bool ref_r1 = maybe_comp(maybe_ref_x, maybe_ref_y);
assert(ref_r0 == ref_r1); // assertion fails
```

This model does not have intuitive value to the developer, nor does it provide a better base from which generic programming with references -- especially as it relates to return types from e.g. `self`-returning member function calls wrapped up -- can behave. The model is consistent, but breaks down because that consistency is applied to the wrong end of the spectrum -- reference semantics -- rather than adhering to C++'s value semantic identity. The salient property of a reference is its value,

not its address, and creating a programming model where the address is the salient property to compare with is likely to lead to confusion and frustration.

Furthermore, note that comparison of a pointer is a strict subset of comparison with a value. For `pointer == pointer`, the answer is true if the addresses are the same and you are dealing with the same object (modulo implementation shenanigans). For `value == value`, all of the same properties of `pointer == pointer` hold, in that if the two objects are than the values are the same. `value == value` for references, despite rebinding for construction and assignment, accurately subsume the `pointer == pointer` model, which means the salient observable properties of an `optional<T&>` are preserved equally -- if not better -- than the a pointer comparison model. What changes is what is denoted as false. In the case of C++, we have an expectation that if objects have the same value representation, they are meant to compare equal : this is an important point. `int` is, for the purposes of regular procedures, identical to `const int&` or even `int&`. They are "semantically equivalent" (Elements of Programming, Page 9). With two values that are the same, the address can be different. This means that for the purposes of a regular procedure, we are treating references as different when they happen to be wrapped in an optional. This is imposing different semantics on a wrapper type that is meant to expose the underlying `T`'s functionality if it is engaged.

This is not a sound programming generic programming model and is in stark contrast with the fundamentals of C++.

4.3.1. But if values are the goal, why not assign-through?

The stark problem with this is the bugs in the mental model from having to fully commit to this paradigm in all aspects of the problem space. This paper does not see a good way to have entirely value semantics or entirely pointer-value (reference) semantics for all operations that is consistent and workable and less prone to bugs for the end user. This is perhaps a demonstration of a greater weakness in C++ with respect to references: the ecosystem and library at large do not support them because they are not regular at all. No strong identity for references were given, save for the small allowance that they do not copy their contents when passed into and returned from subroutines (but do so on raw assignment to and from one another).

A deep comparison in conjunction with a shallow const / rebinding operator is strange to the person who reads all of Elements of Programming and defines the salient properties as the ones that defines how its comparison operators work. There is some room to get away with this from `string_view` since it is a constant view, and thusly some of the properties hold. But, since `optional<T>` is just a wrapper -- and has no promises of Regularity which is the foundation of EoP-style useful properties -- there is a strong case for defining a new kind of `RegularReference` type which is safer and easier to use but still provides useful utility to the end user. Strong comparison has, fortunately, proven most useful.

4.4. The Other Choice

The other choice is, of course, the current status quo: no specialization. Libraries which take this path are llvm::Optional, core::optional, optional lite, absl::optional and the current `std::optional`.

Many of these do so because they claim to implement the C++17 version of `optional` as-is, and try to keep strict conformance with the specification. Most implementations were done around the time of [N3793] or targeted the interface of [N3793] because that was the interface that went into the standard. Many of these implementations also claim to provide C++14/17/etc. features to older compilers (even as far back as C++98 for optional lite): feature-parity (down to the exact same bugs, even) is desirable and necessary for perfect transition and interop between e.g. `absl::optional` and `std::optional`. This unfortunately means that no creativity can be taken with the implementation whatsoever. To quote the library author of core:

> `core` was an attempt to implement proposals to the letter. Because it [(optional references)] didn't make it in, core doesn't do it. — Isabella Muerte of mnmlstc/core, July 4th, 2018

This does not explain *all* optional implementations like this, however. For example, llvm::Optional does not make it a specialization at all, and its implementation predates the version finally ratified in the standard (it was first introduced to LLVM in 2010, but had existed before then as clang::Optional for quite a few more years).

David Blaikie of `cfe-dev` (Clang Front End Development) chimed in about the history of clang::Optional and its successor, llvm::Optional saying that he believes that when there was a need to potentially

push it forward, WG21 had already begun to have serious discussion around such semantics. Because that discussion contained contention about assign-through versus rebind, LLVM/Clang simply decided to not try to introduce the idiom into their code base. They therefore stuck with using `T*` to represent optional reference values in their APIs.

What this ultimately means is that the Standards Body has effectively poisoned the well of discourse and provided a chilling effect on the ecosystem's exploration of optionals with references. By not performing due-diligence on § 4.2 Assign-Through Deployment and Experience, an unverified, untested and undeployed design went from being a mythical unicorn that only existed in theory space to an actual community problem. This is objectively terrible for the C++ community and hopefully this paper brings to light the consequences and reignites a healthier and more grounded discussion based on deployment and implementation experience.

§ 4.5. Pointers?

Pointers have long been heralded as the proper way to have a rebindable optional reference. It's compact in size, already has a `none` value prepared in `nullptr`, and comes with the language itself. It seems to have everything needed. Unfortunately, pointers have problems: chief among them is the requirement that creation of a pointer must be explicit and must come from an l-value. This means that codebases which want to transition from either `boost::optional<const T&>` or from changing a `const T&` parameter to a `T const*` parameter suffer from hard compiler errors at every place of invocation. While this is not a big deal for some functions, it is an incredibly big deal for core APIs.

§ 4.5.1. Temporaries?

Pointers also introduce lifetime and scoping questions and issues: names have to be assigned to temporaries that otherwise had perfect lifetimes that fit exactly the duration of the function call expression: `foo_bar(5);` must become `int temp = 5; foo_bar(&temp);`. While this might be easy to do with integers, it becomes exceedingly complicated for more complex objects and other intricate types. One would have to explicitly control function call lifetime by manually sprinkling curly braces around the call site. This does not scale for older codebases wishing to move themselves to more idiomatic and expressive APIs, or for refactors that

This is not a concern rooted in purely hypothetical thought: 2 survey respondents, a handful of e-mail respondents and several individuals on the CppLang Slack and Discord reported significant pain switching from optional references to `T*`. A programmer responsible for long-term hobby project wrote in:

> Had reference support before upgrading to std::optional, porting those to pointers was quite some work... — Anonymous, July 9th, 2018

There is an observable and significant difference between having to use a pointer and being able to just have the useful lifetime extension rules apply to something that binds to a `const` reference. It is especially painful when one wants to upgrade a function to take a parameter that may or may not have used `const T&` for a parameter that becomes optional. Another way to fix this problem is overloading, but that presents the problem of making it impossible to take the address of a function name without explicitly and directly performing a `static_cast<>`. The only solution that requires absolutely no effort on the part of the programmer is upgrading from `const T&` to `optional<const T&>`, with the caveat that the optional may accept more types than it should. Jonathan Müller talks about this as well in his C++Now 2018 talk 'Rethinking Pointers'.

§ 4.5.2. API Clarity

Pointers with unclear semantics introduce severe user understanding glitches and can lead to programmer bugs due to unclear ownership models. Even programmers in modern C++ struggle deeply with ownership and lifetime, in no small part to the prevalence of pointers in APIs. While LLVM and Clang chose to use pointers for "optional rebindable references", that design decision incurred a huge penalty that the LLVM Community has been paying repeatedly. In some sense, literally: there has been an ongoing offering and a general to-do item to clean up pointer usage in LLVM and Clang as recent as 2017 (the request for using better pointers is gone from the documentation now). Now that they have a well-established convention with clear ownership, it has become harder to mess up.

That unfortunately has not transitioned to all communities.

sol2 has to have a very explicit and loud note in its documentation about how raw pointers are non-owning, complete with a devoted "quick-n-dirty" tutorial entry as well. This has not stopped programmers from shoving raw `new foo();` into entry and exit points to the binding generator and expected magic in terms of ownership reclamation. The Guidelines Support Library is trying to mark all owning raw pointers with a static-analysis capable `owner<T*>` type alias as well as a more fundamental class `non_null<T>`. They also spend time deleting arithmetic operations on the class types, specifically to prevent users from treating single-pointer types like arrays. The presence of the `not_null<T>` class in the Guidelines Support Library and the deletion of the operators on such types in many code bases indicates a world where a plain reference can handle `non_null<T>` is desirable behavior. Unfortunately, types like `non_null<T>` can travel through today's `std::optional<T>`, `std::variant<Ts...>`, and many forms of Standard-anticipating `expected`, while `T&` cannot. This hampers both interoperability (the standard will never create a `non_null<T>`, as known in GSL and as this paper can see it, for the reasonable future) and greatly impedes generic abstractions (invocation-or-error systems with `expected`, delayed invocation systems with `optional`, fire-first result propagation systems with `variant`, and more).

§ 4.5.3. Semantic Quality

A lot of code run under the assumption or behave as if the `T* ptr = nullptr` and a disengaged `optional<T> opt;` are the same and can be used interchangeably. This is, unfortunately, not the case, especially when it comes to working with outside systems. Foreign Function Interfaces to languages and products that already have a concept of `nullptr` or `nil` can induce an extra burden on the system to perform more checks than necessary to check for the many different states involved. For example, consider a mapping between a system and a foreign system which has a notion of the types `T`, `none`, and `nil` for a single object of varying qualification and type:

Semantic Mapping			
Type	**T operation**	**none operation**	**nil/null operation**
`T`	✔	❌	❌
`T&`	✔	❌	❌
`T*`	✔	❌	✔
`optional<T>`	✔	✔	❌
`optional<T&>`	✔	✔	❌
`optional<T*>`	✔	✔	✔

On the tin, this table looks exactly as one would expect. Unfortunately, the reality and the expectation of users is far, far different when it comes to interoperating with the system and how pointers behave. Users -- for APIs like the v8 engine for interfacing with JavaScript, Python, Lua, Squirrel and other foreign system interfaces -- demand that the `none` case not result in a hard error from an API which returns a `T*`. That is, given this class and interface:

```
class foreign_interface {
        template <typename T, typename Key>
        T retrieve(Key&& key) const;
};
```

User's doing `foreign_interface system(...);` and then calling `my_class* p_mc = system.retrieve<my_class*>("some_key");` did not expect failure from the function call if the system only contained `none`. Throwing an error on `none` or invoking panic in this case was deemed actively user hostile and resulted in complaints and raised eyebrows. No matter how hard any library tries, the implicit assumption is `T*` will -- magically -- only be populated with valid data if it exists, but in all other cases just be `nullptr`. It is part of the fact that we continually and perpetually cram more semantics into a single `T*` than any other type in C and C++. Pointers are a useful and powerful tool, but they have become an omnibus for communicating things at the type system level to the point that they become effectively useless in conveying proper semantics, other than "DANGER HERE".

This was the source of a deep division in the community as well over how `nullptr` in `string_view` should be handled, see [p0903r2]. `nullptr` was not supposed to be part of the semantics for `string_view` but, invariably, it became a point of contention because the mental model for `const char*` included `nullptr` as a sentinel, not just an empty-sized `string_view`. Unfortunately, many C APIs conflated the two for decades and so pointers -- even pointers + size -- are invariably tainted by the discourse, resulting in a new table:

Semantic Mapping

Type	T operation	none operation	nil/null operation
`T`	✔	❌	❌
`T&`	✔	❌	❌
`T*`	✔	✔	✔
`optional<T>`	✔	✔	❌
`optional<T&>`	✔	✔	❌
`optional<T*>`	✔	✔	✔

It is unfortunate that `optional<T*>` and `T*` in many user-facing APIs will essentially result in the same level of work for foreign system interfaces like these due to legacy mental models of what a pointer is and end-users cutting themselves on sharp edges. `empty`/`none` and `nullptr` are conflated in a way that only `optional<T&>` and `T*` are capable of fixing: the lack of this has resulted in clear overall loss for the community in general. If each of those columns was its own independent API check, then users relying on `T*` -- regardless of whether they are aware of the fine split between `nil`/`nullptr` and `none` -- get robbed of both efficiency (in checking only the minimal subset of things that they need to) and API space.

While the first table in this section makes it clear that an API can -- and, maybe should -- differentiate between `nil` and `none`, the reality of the matter is that pointers have massively overloaded meaning in all number of APIs and therefore backwards compatibility and ease of handling user understanding requires something different than a pointer due to its shared knowledge burden.

§ 5. Recommendations

Originally, this proposal laid out several solutions in hopes that the community would test each of their merits, providing a natural evolution towards a correct choice. Unfortunately, failure to gain consensus in the Committee during the San Diego 2018 has ruled out a conservative consensus and continued unfounded and bad faith advocacy without due diligence has made it impossible to maintain a neutral and community-guided discourse. Chief among the concerns was not fully defining all operations and that a type was for a template which originally had stronger semantics was not a good design. Given the discouragement of the conservative solution but the encouragement in continued scholarship and effort in determining the right semantics, the work has been done here to come up with the following recommendation.

§ 5.1. Recommendation: Rebinding, shallow const, deep comparison reference types

This is the "complete" solution that is seen as a step up from the conservative solution. It is the version that has seen adoption in `boost::optional` for over 15 years: it is a rebinding optional reference with `my_op = my_lvalue;` being a valid expression.

Rebind semantics have the benefit of having no surprises in engaged vs. unengaged states, as shown in § 3.1 The Great Big Table of Behaviors.

`boost::optional` is not the only implementation of rebinding optionals. There are over 4 publicly available (and highly regarded) optional implementations that have references and behave in this fashion. It is the typical community choice when one is starting a new project, and has been a staple for many years. Among its design is the chief semantic that certain code will always be wrong, no matter what. Consider the code from § 4.2 Assign-Through Deployment and Experience:

```
int foo_bad(int x, optional<int&> maybe_y) {
        int value = 40;
        /* lots of code */
        maybe_y = value;
        /* ... */
        return 24;
}
```

Now, the moment anyone sees `maybe_y = value;` (including lifetime analysis tools like Clang's experimental `-Wlifetime` and the in-the-works GCC lifetime analyzer), it can easily detect this as an error because the semantics do not change based on a runtime property. R-value construction is a bit harder to outright ban for `optional<const T&>`, which is similar to `string_view` in its complexities. `optional<const T&>` construction can be made valid (for the purposes of parameter passing, which is important for code base improvement and migration), but normal assignment from an r-value

becomes a hard compiler error. There is still the case of `std::optional<const T&> foo = T{};` not being a hard compiler error if implicit construction is allowed, and there's no mechanism that can be deployed in writing the `optional` constructors and assignment operators that will ban this piece of code while still allowing it for parameters. This paper note that the community lives with these same problems for `string_view` and may have to live with them for the upcoming `function_ref` as well.

This paper also recommends that comparisons should be "deep", in that they compare the values and not the identities of the reference. This model has widespread industry usage and practice, and has so far not caused undue problems for the C++ community measurable from any user feedback or in their use of generic algorithms. While individuals fixated with `Regular` and the fundamentals of generic programming will find this (mildly?) distasteful, an optional reference behaving more like the already-standardized constant view types appears to have no great detrimental effect on algorithms in either the standard or outside code bases.

On top of this, the goal is to make the `optional` object itself consistent as a wrapper, not what it wraps. Many people point to the specification of `operator=` as an example where the "details of `T` leak into the `optional`", stating that because an assignment from `T` can affect the "held value" in a value-holding `optional<T>`, it "presents the mental model of being exactly like `T`". This is absolutely not the case: a `std::string` isn't a `std::string_view` because it can be assigned from one, nor is a `std::string` a `std::initializer_list<char>`. The assignment operation behaves like this -- primarily -- because the community, the original author, and the Committee is not trying to create sub-optimal code for the sake of theoretical consistency.

To illustrate, there is a library called `rangeless` that has its own `maybe<T>` type. Rather than have an `operator=` that performs the smart thing, for the sake of "consistency" it performs a `reset();` followed by a `new (memory()) T(std::move(rhs))` for its operator= and reset(val) calls. If this `maybe<T>` were to contain a `std::vector<T>`, this type would not only force destruction of the original vector, but a whole new memory allocation even if the incoming vector was smaller in size than the old one, and even if its move constructor would just steal its guts. This is not good behavior, and hand-wringing about consistency in the assignment operator and trying to work around it (or delete it) leads to suboptimal code and absolutely ridiculous semantics that sacrifice performance and correctness for the sake of a purity or consistency that serves nobody better than before.

`std::optional<T&>` being assign-able from `T&` is not an abstraction leak, or "trying to model `T` whenever possible": it is a natural, performant, and necessary part of the abstraction that retains performance and ease of use at no cost to generic and general code. Algorithms and generic code working with an `optional` object is not broken because it acknowledges its `optional`-ness, and a reference optional that rebinds on assign preserves that the object -- the `optional<T>` -- is being changed, but that its internal invariants are its business, is exactly in line with a generic abstraction of "maybe T, maybe Not".

§ 6. Acknowledgements

Thank you James Berrow, whose work reminded me that thorough scholarship is always the right and sound choice to make.

§ References

§ Informative References

[ABSEIL-OPTIONAL]
Titus Winters; Google. abseil. July 4th, 2018. URL: https://github.com/abseil/abseil-cpp

[AKRZEMI-OPTIONAL]
Andrzej Krzemieński. Optional (nullable) objects for C++14. April 23rd, 2018. URL: https://github.com/akrzemi1/Optional

[ARGOT]
Matt Calabrese. argot. July 1st, 2018. URL: https://github.com/mattcalabrese/argot

[BOOST-OPTIONAL]
Fernando Luis Cacciola Carballal; Andrzej Krzemieński. Boost.Optional. July 24th, 2018. URL: https://www.boost.org/doc/libs/1_67_0/libs/optional/doc/html/index.html

[FOLLY-OPTIONAL]
Facebook. folly/Optional. August 11th, 2018. URL: https://github.com/facebook/folly

[FOONATHAN-CPPNOW]
Jonathan Müller. C++Now 2018: Rethinking Pointers. May 9th, 2018. URL: https://foonathan.net/cppnow2018.html

[FOONATHAN-OPTIONAL]
Jonathan Müller. type_safe. June 22nd, 2018. URL: https://github.com/foonathan/type_safe

[FOONATHAN-OPTIONAL-PROBLEMS]
Jonathan Müller. Let's Talk about std::optional<T&> and optional references. July 12th, 2018. URL: https://foonathan.net/blog/2018/07/12/optional-reference.html

[HEISENBUGS]
Heisenbugs. August 20th, 2018. URL: https://en.wikipedia.org/wiki/Heisenbug

[LLAMA-OPTIONAL]
Sy Brand (TartanLlama). Optional. June 7th, 2018. URL: https://github.com/TartanLlama/optional

[LLVM-OPTIONAL]
LLVM Developer Group. Optional.h. July 4th, 2018. URL: http://llvm.org/doxygen/Optional_8h_source.html

[LLVM-OPTIONAL-HISTORY]
David Blaikie. [clang::Optional] History Digging. July 10th, 2018. URL: http://lists.llvm.org/pipermail/cfe-dev/2018-July/058448.html

[MARTINMOENE-OPTIONAL]
Martin Moene. Optional Lite. June 21st, 2018. URL: https://github.com/martinmoene/optional-lite

[MNMLSTC-OPTIONAL]
Isabella Muerte. core::optional. February 26th, 2018. URL: https://mnmlstc.github.io/core/optional.html

[N3793]
Fernando Luis Cacciola Carballal; Andrzej Krzemieński. A proposal to add a utility class to represent optional objects (Revision 5). March 10th, 2013. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3793.html

[OPTIONAL-SURVEY]
Optional: What's In Our Codebases. August 20th, 2018. URL: https://github.com/ThePhD/future_cxx/blob/04370836748fcae65f898a817a9977a466a8ac6f/references/raw/optional__%20Survey.xlsx

[P0798]
Sy Brand. Monadic operations for std::optional. May 4th, 2018. URL: https://wg21.tartanllama.xyz/monadic-optional/

[P0903R2]
Ashley Hedberg, Titus Winters, Jorg Brown. Define basic_string_view(nullptr). 7 May 2018. URL: https://wg21.link/p0903r2

[P1175R0]
JeanHeyd Meneide. A simple and practical optional reference for C++. 6 October 2018. URL: https://wg21.link/p1175r0

[SOL2]
ThePhD. sol2: C++ <-> Lua Binding Framework. July 3rd, 2018. URL: https://github.com/ThePhD/sol2

[TCBRINDLE-POST]
Tristan Brindle. The Case for Optional References. September 16th, 2018. URL: https://tristanbrindle.com/posts/optional-references